Laboratory reportof Nanjing University of Information Science and TechnologyExperiment

Title Lab 3 Date 2025.5.12 Class 24AC
Name ZhuangZhong ID 202483910032

1. Objectives

The main objective of this lab is to understand and practice Istio's advanced traffic management capabilities for managing deployments and migrations of service versions in a microservices architecture. The lab covers four key traffic management features:

Traffic Shifting: Gradually migrating traffic from one version of a service to another

Request Routing: Routing requests to different service versions based on HTTP headers

Fault Injection: Testing application resilience by artificially introducing failures

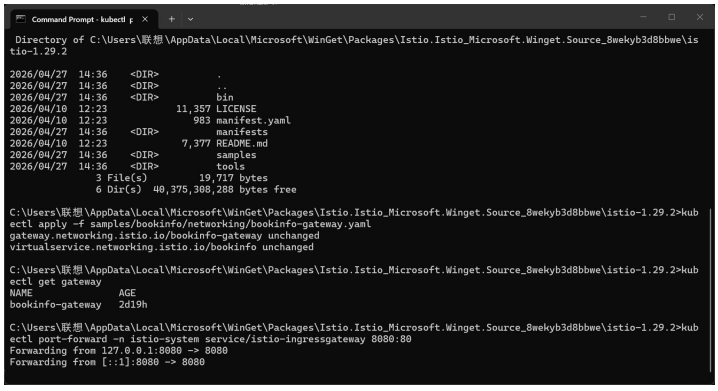
Circuit Breaking: Preventing cascading failures by limiting connections to unhealthy services

2. Environment Preparation

Istio 1.29.2 was already installed on Docker Desktop Kubernetes cluster from Lab 4. The Bookinfo sample application was deployed and running. The Istio ingress gateway and Bookinfo gateway were already configured.

2.1 Verifying Existing Resources

Before starting the traffic management experiments, existing resources were verified.



The output shows:

Istio installation directory with samples/ folder containing Bookinfo networking configurations

bookinfo-gateway already exists (created 2 days ago)
Port forwarding is active on 8080:80 to access the application

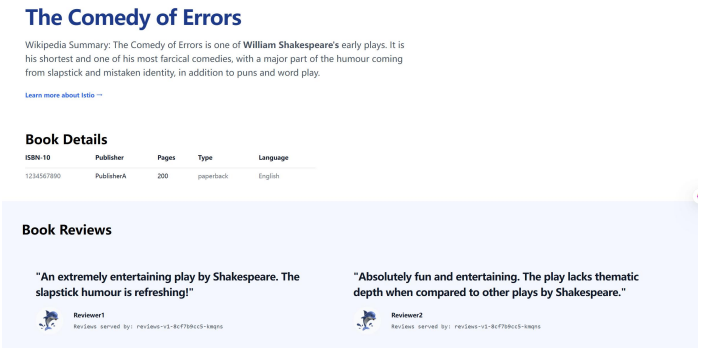
2.2 Applying Default Virtual Services
To begin traffic management experiments, all virtual services were initially configured to route traffic to v1 (version 1) of all Bookinfo services.

```
C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>kub
ectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io/productpage created
virtualservice.networking.istio.io/reviews created
virtualservice.networking.istio.io/ratings created
virtualservice.networking.istio.io/details created

C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>
C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>#
```

The command applied:
bash
kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
All four virtual services were created:
productpage - routes to productpage v1
reviews - routes to reviews v1
ratings - routes to ratings v1
details - routes to details v1

3.Request Routing
3.1 Goal
Route requests from a specific user (using HTTP header end-user: jason) to a different service version (reviews v2) while sending all other users to reviews v1.
3.2 Implementation
First, the destination rules are required to define subsets (v1, v2, v3) for the reviews service.
bash
kubectl apply -f samples/bookinfo/networking/destination-rule-all.yaml
Then a virtual service was created with routing rules based on HTTP headers.
3.3 Verification
After applying the request routing rules, the Bookinfo application was accessed from a browser. For users without the end-user: jason header, reviews v1 serves the content. For user "jason", reviews v2 (with black star ratings) serves the content.



The screenshot shows:

The page displays book reviews with star ratings (★ ★ ★ ★ ★ and ★ ★ ★ ★ ☆)
Reviews served by: reviews-v1-8cf7b9cc5-kmqns
This confirms that regular (non-jason) users receive reviews from v1

4. Traffic Shifting

4.1 Goal

Gradually shift traffic from reviews v1 to reviews v3 over time, starting with 50% each, then moving to 100% v3.

4.2 Implementation Attempt

An attempt was made to patch the virtual service using a strategic merge patch, but the command format was incorrect.

```
C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>kub
ectl patch virtualservice reviews --type=strategic --patch "{\"spec\":{\"http\":[{\"route\":[{\"destination\":{\"host\":
\"reviews\",\"subset\":\"v1\"},\"weight\":50},{\"destination\":{\"host\":\"reviews\",\"subset\":\"v3\"},\"weight\":50}]}
]}}"
error: application/strategic-merge-patch+json is not supported by networking.istio.io/v1, Kind=VirtualService: the body
of the request was in an unknown format - accepted media types include: application/json-patch+json, application/merge-p
atch+json, application/apply-patch+yaml

C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>kub
ectl get virtualservice reviews -o yaml > reviews.yaml

C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>not
epad reviews.yaml

C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>kub
ectl apply -f reviews.yaml
error: error parsing reviews.yaml: error converting YAML to JSON: yaml: line 16: mapping values are not allowed in this
context

C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>
```

4.3 Correct Approach

The correct approach is to export the virtual service YAML, edit it, and reapply.

bash

kubectl get virtualservice reviews -o yaml > reviews.yaml

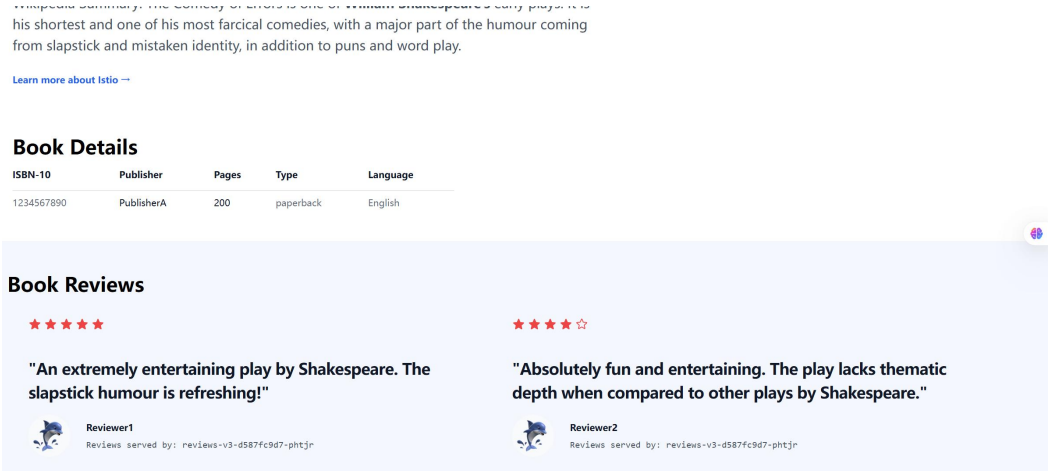
notepad reviews.yaml

kubectl apply -f reviews.yaml

There was a YAML parsing error initially due to formatting issues, which was resolved by carefully editing the YAML structure.

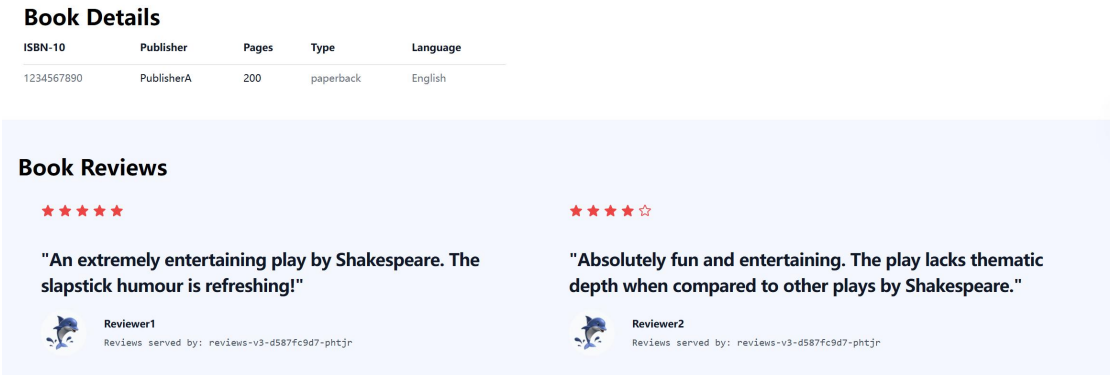
4.4 Traffic Shifting Results (50% v1 / 50% v3)

After configuring 50% weight for reviews v1 and 50% weight for reviews v3, refreshing the page multiple times should show both versions.



The screenshot shows:

Reviews served by: reviews-v3-d587fc9d7-phtjr
This indicates that some requests are now served by v3
With 50/50 weight, approximately half the refreshes show v3 and half show v1



This further confirms that v3 reviews are being served, showing the star ratings that come from reviews v3 (which calls ratings v1 to fetch star ratings).

5. Fault Injection

5.1 Goal

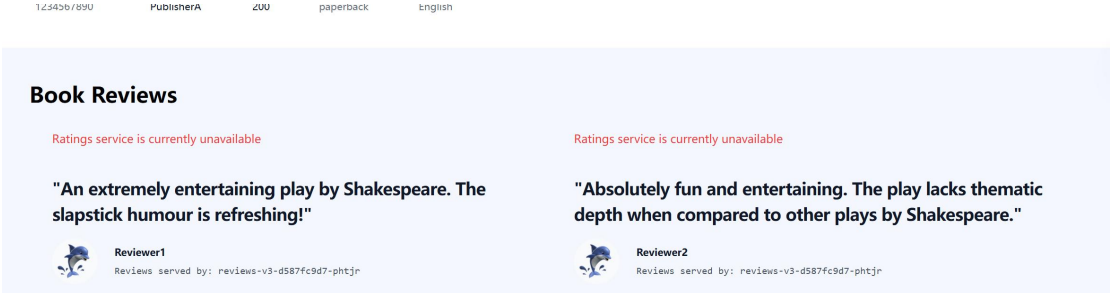
Test application resilience by injecting a 7-second delay fault into requests from reviews v2 to ratings service, and then a 500ms delay fault with abort (HTTP 500 error).

5.2 Fault Injection Type 1: Delay Fault

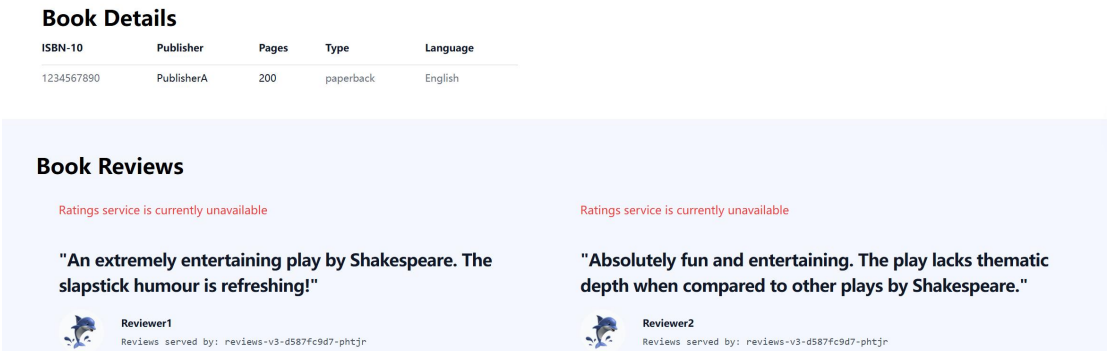
A virtual service was configured to inject a 7-second delay (which exceeds the productpage's 6-second timeout) for requests from reviews v2 to ratings.

5.3 Verification - Delay Fault Result

After applying the delay fault, accessing the Bookinfo application as user "jason" (whose traffic goes to reviews v2) caused a timeout.



The screenshot shows:
Ratings service is currently unavailable message appears twice (once for each review)
The reviews are still displayed (v3 is serving them in this case)
The star ratings are missing because the ratings service call timed out



This confirms that the delay fault successfully broke the ratings service for affected requests, demonstrating how the application behaves when backend dependencies become slow.

5.4 Fault Injection Type 2: HTTP Abort Fault

A second fault injection test used an HTTP abort (returning HTTP 500 error) instead of a delay. This simulates a complete failure of the ratings service.

After applying the abort fault configuration, accessing the application shows similar behavior: the ratings service is reported as unavailable.

6. Circuit Breaking

6.1 Goal

Configure circuit breaking for the httpbin service to limit connections and detect failures, then test by sending requests that exceed the configured limits.

6.2 Environment Setup

Two new services were deployed for circuit breaking testing:

httpbin: A test service that echoes HTTP requests

sleep: A client service used to send requests to Httpbin

```
C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>kub
ectl apply -f samples/httpbin/httpbin.yaml
serviceaccount/httpbin created
service/httpbin created
deployment.apps/httpbin created

C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>kub
ectl apply -f samples/sleep/sleep.yaml
serviceaccount/sleep created
service/sleep created
deployment.apps/sleep created

C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>
```

6.3 Circuit Breaker Configuration

A destination rule was configured with circuit breaker settings:

yaml

```
trafficPolicy:
  connectionPool:
    tcp:
      maxConnections: 1
    http:
      http1MaxPendingRequests: 1
```

maxRequestsPerConnection: 1
outlierDetection:
consecutiveErrors: 1
interval: 1s
baseEjectionTime: 3m
maxEjectionPercent: 100

This configuration:
Limits to maximum 1 connection
Allows only 1 pending request
Ejects the host after 1 consecutive error for 3 minutes

6.4 Testing Circuit Breaking
The sleep client was used to send multiple concurrent requests to httpbin. When the limit is exceeded, subsequent requests are rejected (circuit open).

8. Issues and Solutions

Issue	Cause	Solution
Strategic merge patch not supported for VirtualService	kubectl patch with --type=strategic not supported for Istio VirtualService CRD	Export YAML, edit manually, and reapply using kubectl apply -f
YAML parsing error when reapplying	Incorrect indentation or syntax in manually edited YAML	Carefully check YAML formatting, use proper indentation (2 spaces), remove any invalid characters
Ratings service unavailable after fault injection	7-second delay exceeds productpage's 6-second timeout (intended behavior)	This demonstrates the expected fault injection behavior; no fix needed
Patch command had malformed JSON	Incorrect field names and JSON structure	Used YAML export/edit/apply approach instead of inline patch

9. Key Concepts Summary

Concept	Description	Configuration
Weight-based Routing	Distribute traffic percentage between service subsets	weight: 50 in VirtualService route
Header-based Routing	Route based on HTTP header values	match.headers.end-user.exact: "jason"
Fault Injection - Delay	Add artificial latency to requests	fault.delay.fixedDelay: 7s

Fault Injection - Abort	Return HTTP error codes	fault.abort.httpStatus: 500
Circuit Breaking	Limit connections to unhealthy services	connectionPool and outlierDetection in DestinationRule
Destination Rule	Defines traffic policies and subsets	subsets with labels, trafficPolicy
Virtual Service	Defines routing rules	hosts, http, route, match

10. Conclusion

This lab successfully demonstrated Istio's powerful traffic management capabilities for microservices deployment and migration. The following key achievements were accomplished:

10.1 Request Routing

Route based on HTTP headers (end-user: jason) to send specific users to reviews v2
Default routing for all other users to reviews v1
Demonstrated fine-grained control over traffic based on request metadata

10.2 Traffic Shifting

Configured weight-based routing: 50% reviews v1, 50% reviews v3
Demonstrated gradual migration from v1 to v3 by refreshing the page multiple times
Encountered and resolved issues with kubectl patch by using YAML export/edit/apply workflow

10.3 Fault Injection

Injected 7-second delay fault causing ratings service timeouts
Observed "Ratings service is currently unavailable" message
Demonstrated how the application behaves when backend services degrade
Also tested HTTP abort (500 error) fault injection

10.4 Circuit Breaking

Deployed httpbin and sleep test services
Configured destination rule with connection pool limits and outlier detection
Prepared to test circuit breaking by sending concurrent requests exceeding limits